

1. System Architecture & Context Bound

- **Core Logic:** The SQL Generation logic is handled natively by our core LLM orchestration layer (LangGraph). The LLM processes the database structure to synthesize raw SQL text parameters.
- **The Plugin Interface:** The Model Context Protocol (MCP) server acts as the secure execution interface, exposing exactly two tools to the LLM.

2. Exposed MCP Tool Contracts

Tool 1: describe_schema

- **Description:** Scans the PostgreSQL system catalogs to build a structural metadata map of the database. This allows the LLM to understand what data is available without fetching live records.
- **Input Parameters:** None
- **Output Data:** A structured JSON object dictionary mapping tables to their respective column names and data types.
- *Example Output:* {"products": {"product_id": "integer", "product_name": "varchar"}}

Tool 2: query_database

- **Description:** Accepts a compiled SQL text string from the LLM, executes it against the targeted database instance via psycopg2, and returns the active records.
- **Input Parameters:** sql (string, Required): A syntactically valid SQL text statement.
- **Output Data:** A list of dictionaries, where each dictionary represents a row of data retrieved.
- *Example Output:* [{"product_name": "Chai", "revenue": 1420.50}]

3. Built-In Security Guardrail Design

To guarantee a strict read-only environment, an automated interceptor function is built directly into the entry gate of the query_database tool:

- **Enforcement Mechanism:** Every incoming sql string parameter is instantly scanned by a local Python regex validator before a database connection is opened.
- **Banned Keywords:** INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE.
- **Violation Behavior:** If any banned keyword is detected, the transaction is dropped immediately, execution halts, and a clean access-denied message is returned to the user interface.